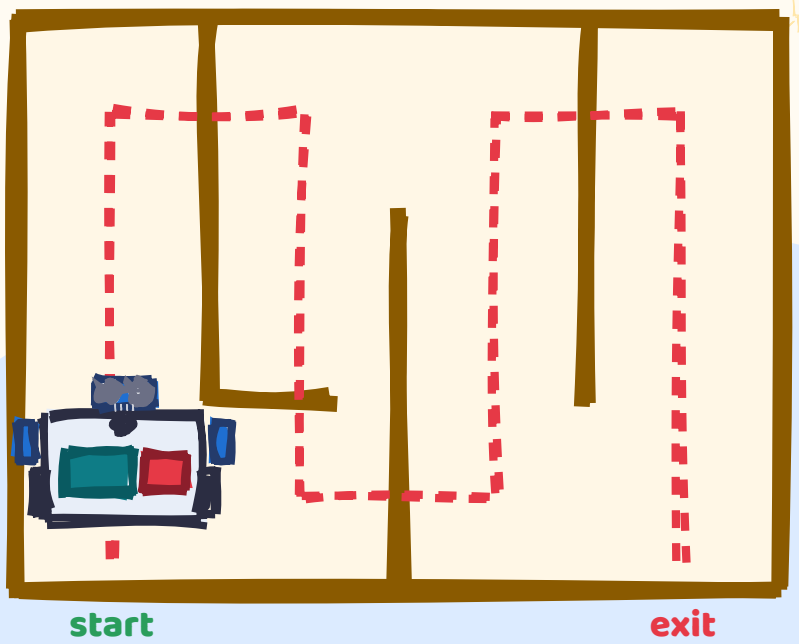
Tech Leaders · Class 2

Autonomous Maze Solving Robot

A robot that finds its own way out, using PID control

3 ultrasonic sensors | PID wall following | Left-hand rule



Did you know?
PID control steers cruise control, drones, 3D printer heaters and rockets. You will tune one today.

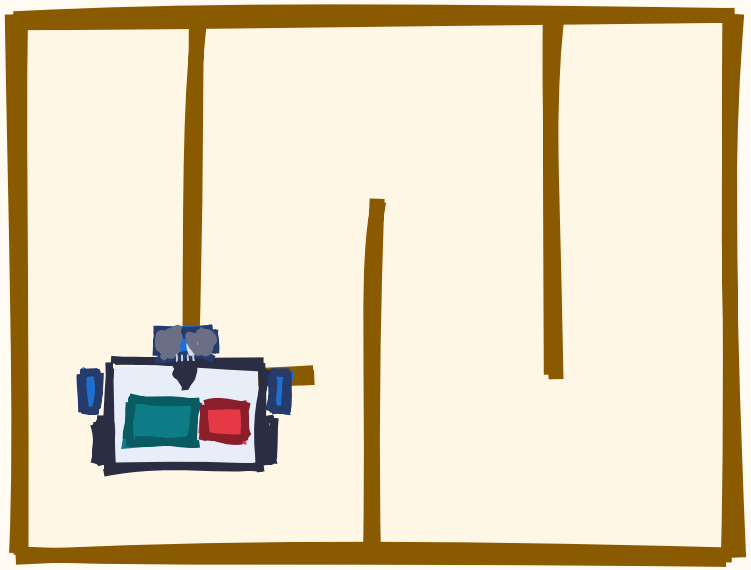
LEARN

Engineering challenge

No map, no GPS, no remote control

Rescue robot

After an earthquake, rescuers send small robots into damaged buildings. There is **no map**, GPS does not work indoors, and radio signals are weak. The robot must **find its own way**, keeping a safe distance from walls.



Drive it by remote?

Bump into walls?

Sense and correct!

Today: you build a robot that follows walls smoothly using **PID control** and solves a maze with the **left-hand rule**.

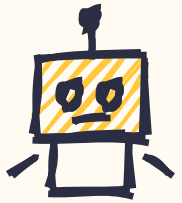
Learning aims

Today I will learn to...

1. Measure distance with **three ultrasonic sensors** without crosstalk
2. Drive two motors forwards and backwards with the **L298N**
3. Explain and code **P, I and D** feedback control
4. **Tune** Kp, Ki and Kd with a fair, recorded method
5. Solve a maze with the **left-hand rule** and compare times

Keywords

feedback · error · PID · Kp Ki Kd · overshoot · oscillation · wall following · algorithm



LEARN

Watch and think

Video time!



Arduino Maze-Solver and Wall-Follower Robot

Ariel Bernal · about 4 minutes · youtu.be/SFfYNPrc6c4

Watch for...

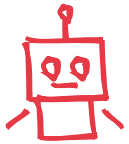
1. Which wall does the robot keep next to it?
2. What does it do at a **dead end**?
3. How does it find a **shorter** path on the second run?

Scan to watch

After the video: Draw a small maze. Trace the left-hand rule with your finger. Does it always reach the exit?

Real world

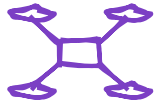
Where PID and wall-following are used



Rescue robots
Explore collapsed buildings and tunnels



Lane keeping
Cars correct their steering to stay centred



Drones
Hold their height and angle against the wind



Robot mowers
Follow edges and boundaries

? Micromouse is a world-wide competition where tiny robots race through mazes. Why do the fastest robots need PID?

Fun fact
Your bike riding is PID: you notice the error and correct it smoothly.

Big idea

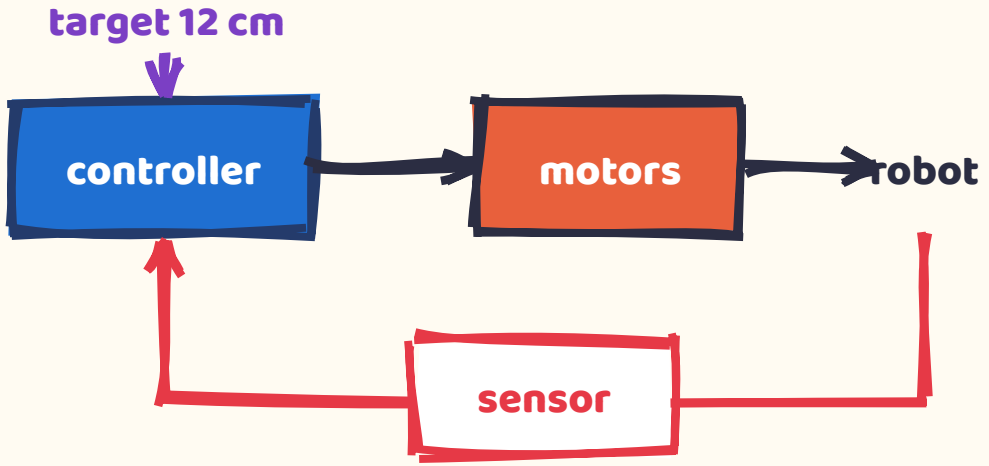
Open loop vs closed loop

Open loop (guessing)

'Drive forward for 2 seconds.' If one motor is slightly weaker, the robot drifts into the wall and **never knows**.

Closed loop (feedback)

Measure the distance to the wall **many times a second**, compare it with the target, and correct the steering. Errors are noticed and fixed.



Error

error = target - measured. Everything in PID is built from this one number.

The maths

P, I and D in plain words

P

Proportional: the present

Steer harder the bigger the error. Too much and the robot zig-zags.

```
Kp × error
```

I

Integral: the past

Adds up old errors to remove a small steady offset. Too much and it overshoots slowly.

```
Ki × sum of errors
```

D

Derivative: the future

Looks at how fast the error is changing and brakes the correction. Calms the zig-zag.

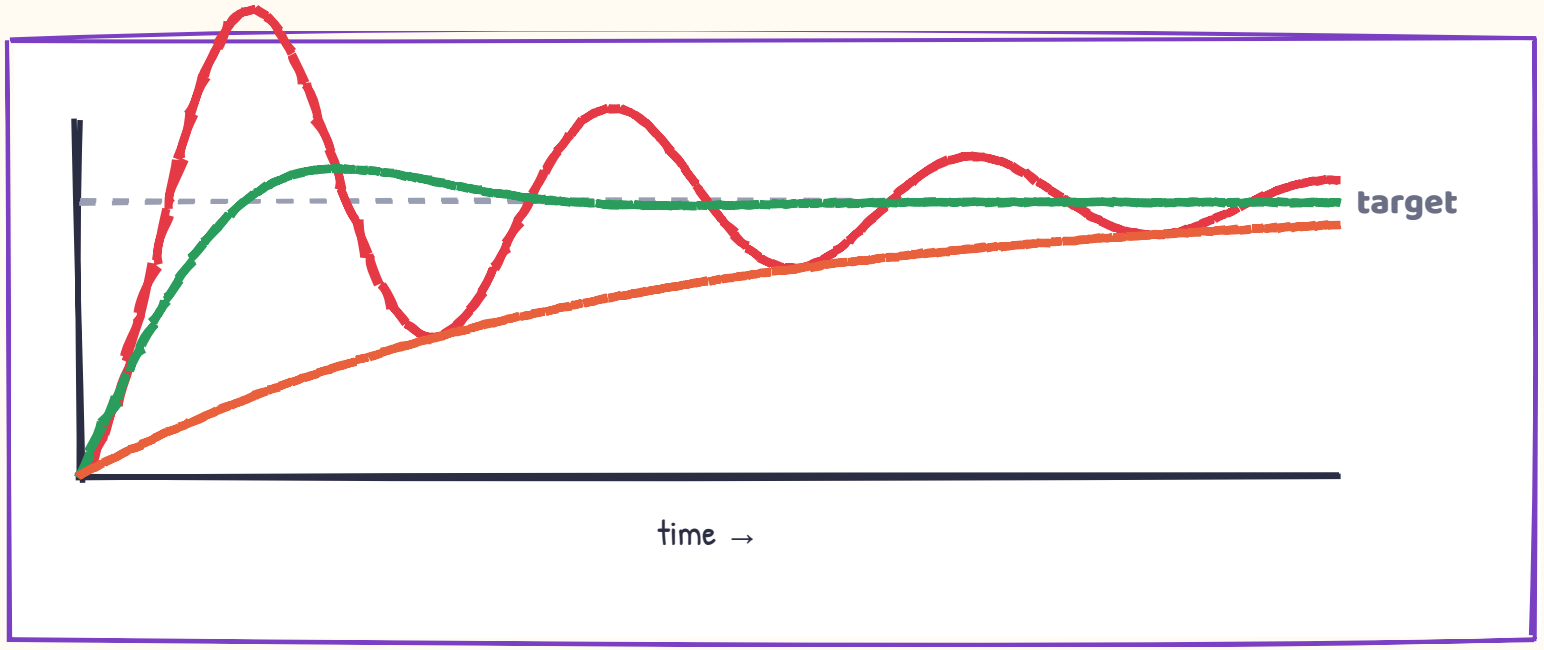
```
Kd × (error - last error)
```

correction = Kp·e + Ki·Σe + Kd·Δe → left motor = base + correction, right motor = base - correction

LEARN

What tuning looks like

Three robots, three behaviours



Kp too high

Overshoots and oscillates: zig-zags down the corridor.

Kp too low

Slow and lazy: drifts and bumps into corners.

Good P + D

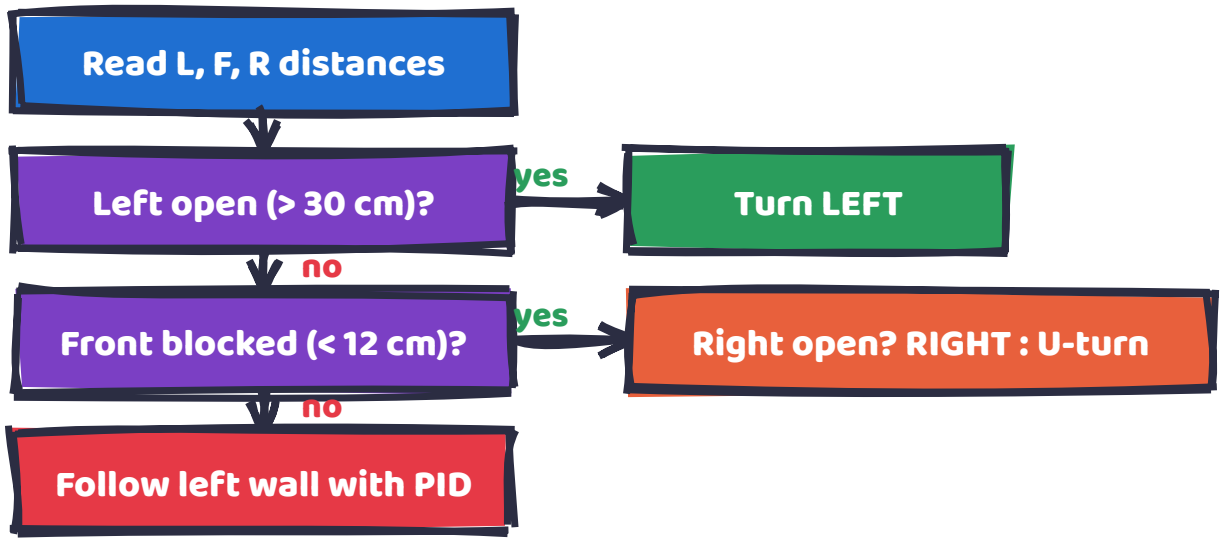
Reaches the target quickly with little overshoot.

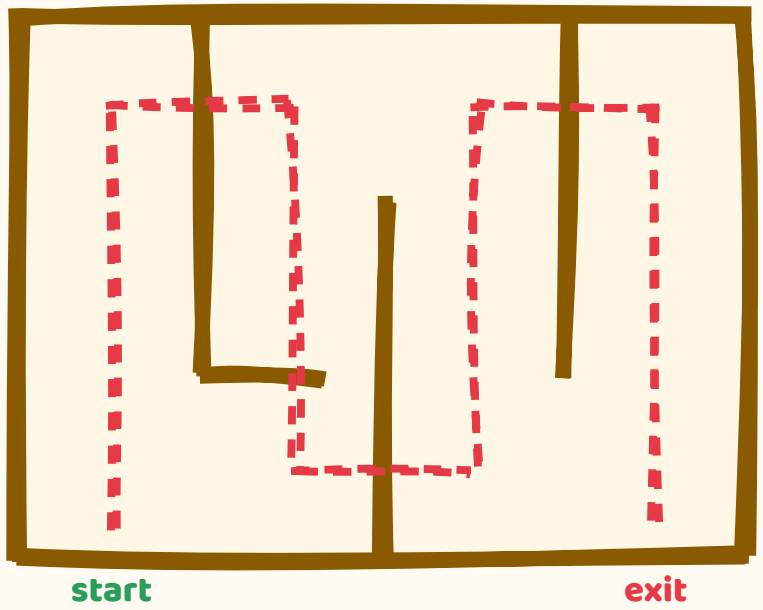
Plot it!

Print the error to the Serial Plotter with the wheels lifted, then move a book closer and further. You will SEE these curves.

The algorithm

Left-hand rule: keep your hand on the wall





Why it works

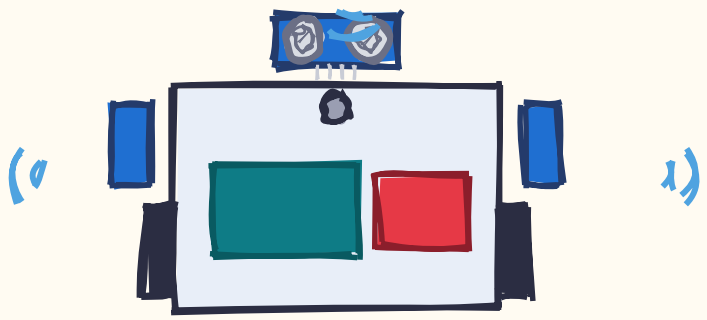
In a maze with no loops, keeping one hand on the wall always leads to the exit. It is not the shortest path, but it never gets lost.

Priority matters

Check LEFT first, then FRONT, then RIGHT. Change the order and you get the right-hand rule.

Sensing

Three ultrasonic eyes



Distance maths

Sound travels about 343 m/s. The echo time covers the trip there **and back**, so distance in cm = echo µs ÷ 58.

Sensor	TRIG	ECHO
Left	A0	A1
Front	A2	A3
Right	10	11

Crosstalk!

Three sensors firing together hear each other's echoes. Read them **one at a time** with a short pause between.

Analog pins?
A0–A5 work as normal digital pins too. Handy when you run out!

Mounting
Side sensors exactly 90° to the body. A tilted sensor measures the wrong distance.

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Kit per group
2. Assemble the chassis
3. Wire the motor driver
4. Test the motors
5. Mount the three sensors
6. Build the class maze
7. Pins, sensors and motors

Build it

Kit per group



2WD robot chassis
motors, wheels, castor



L298N motor driver
ENA / ENB jumpers removed



3 × HC-SR04
left, front, right



Arduino Uno
+ USB cable



4 × AA pack
motor power



Screwdriver set
+ cable ties, tape

Also: M-F and M-M jumpers, a USB power bank for the Uno, cardboard maze walls (class), laptop with Arduino IDE 2, stopwatch.

Double session
Plan 2 × 90 min: build + motor test, then PID tuning + maze race.

Build step 1 of 5

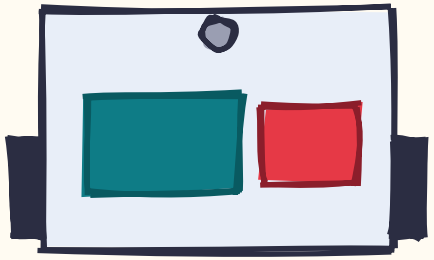
Assemble the chassis

a Fix the two motors with the metal brackets. Wires point inwards. Push the wheels firmly on the shafts.

b Fit the castor at the front. Check the chassis sits level and the wheels spin freely.

c Mount the Uno and the L298N with screws or double-sided tape, away from the wheels.

d Fit the 4 × AA pack where you can reach the switch.



Check: the robot rolls straight when pushed gently by hand.

Do not over-tighten screws into acrylic: it cracks.

Build step 2 of 5

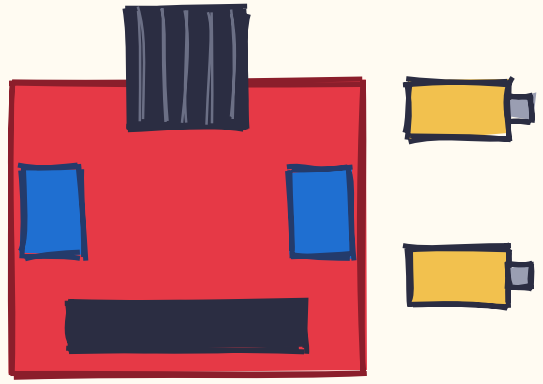
Wire the motor driver

a Left motor → **OUT1/OUT2**, right motor → **OUT3/OUT4**.

b Remove the **ENA and ENB jumpers**. ENA → pin 5, IN1 → 4, IN2 → 7, ENB → pin 6, IN3 → 8, IN4 → 12.

c Battery **+** → **12V** terminal, battery **-** → **GND** terminal, and a wire from that GND to the Uno **GND** (common ground!).

d Power the Uno from USB (laptop or power bank), not from the L298N 5V pin: 6 V in is too low for its regulator.



L298N driver

Check: the L298N power LED lights when the battery switch is on.

No common ground = motors ignore the Uno. It is the number 1 wiring mistake.

Build step 3 of 5

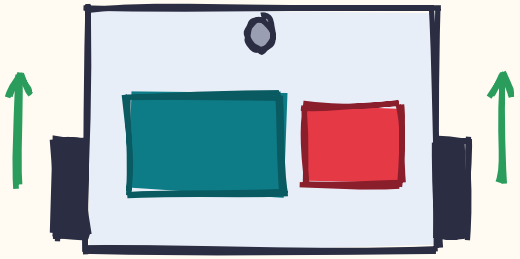
Test the motors

a Upload the **drive()** test from part 1 of the code with the wheels **lifted off the table**.

b Call **drive(150, 150)**: both wheels must spin forwards. If one spins backwards, swap that motor's two wires.

c Try drive(150, -150): the robot spins on the spot. Note which way.

d Find the **lowest speed** that still moves the robot on the floor (often 90-120). That is your minimum.



Check: both wheels forward at drive(150,150) and the robot drives straight-ish on the floor.

MAKE

Build step 4 of 5

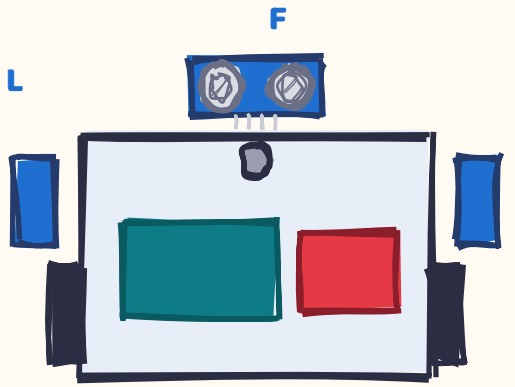
Mount the three sensors

a Front sensor in the middle, facing forward. Left and right sensors facing exactly sideways, at the same height.

b Wire each with VCC → 5V, GND → GND, and TRIG/ECHO as in the pin table (A0-A3, 10, 11).

c Upload a test that prints L, F, R in cm. Hold a book at 10, 20 and 30 cm from each sensor.

d Mark each sensor's cable with tape: L, F, R. Future you will thank you!



Check: every sensor reads within about 1-2 cm of a ruler.

Readings of 0 or 400? Check TRIG/ECHO are not swapped.

Build step 5 of 5

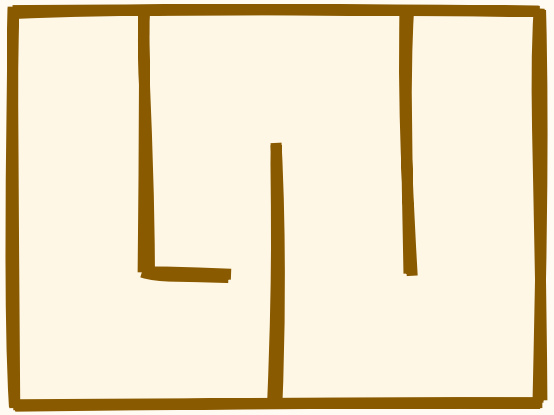
Build the class maze

a Use cardboard walls at least **15 cm tall** (taller than the sensors) and corridors about **30 cm wide**.

b Tape walls firmly to the floor. Corners must be square: gaps confuse the sensors.

c Start with one straight corridor for PID tuning, then add left and right turns and dead ends.

d Keep the maze **loop-free** so the left-hand rule always works.



Check: a robot-sized box can travel through every corridor without touching the walls.

Soft cloth or angled walls absorb ultrasound. Flat cardboard works best.

Code it: part 1

Pins, sensors and motors

```
const int ENA = 5, IN1 = 4, IN2 = 7;   // left motor
const int ENB = 6, IN3 = 8, IN4 = 12;  // right motor
const int TL = A0, EL = A1, TF = A2, EF = A3, TR = 10, ER = 11;

void setup() {
  int outs[] = {ENA, IN1, IN2, ENB, IN3, IN4, TL, TF, TR};
  for (int p : outs) pinMode(p, OUTPUT);
  pinMode(EL, INPUT); pinMode(EF, INPUT); pinMode(ER, INPUT);
  Serial.begin(9600);
}

long readCm(int trig, int echo) {
  digitalWrite(trig, LOW);  delayMicroseconds(2);
  digitalWrite(trig, HIGH); delayMicroseconds(10);
  digitalWrite(trig, LOW);
  long us = pulseIn(echo, HIGH, 25000);   // give up after 25 ms
  if (us == 0) return 400;                // no echo = far away
  return us / 58;
}

void drive(int left, int right) {          // -255 ... 255
  digitalWrite(IN1, left >= 0);  digitalWrite(IN2, left < 0);
  analogWrite(ENA, constrain(abs(left), 0, 255));
  digitalWrite(IN3, right >= 0); digitalWrite(IN4, right < 0);
  analogWrite(ENB, constrain(abs(right), 0, 255));
}
```

pulseIn timeout

Without a timeout, a missing echo freezes the robot for a whole second. 25 ms covers about 4 m.

drive(left, right)

Positive = forward, negative = backward. One function for every move: forward, turn, spin.

Test first

Use only setup() + drive() to check motor directions before adding PID.

Code it: part 2

PID wall following + left-hand rule

```
float Kp = 4.0, Ki = 0.0, Kd = 1.5;   // start values, then tune
const float TARGET = 12;              // cm from the left wall
const int BASE = 150;
float integral = 0, lastError = 0;

void loop() {
  long L = readCm(TL, EL); delay(10);    // one at a time:
  long F = readCm(TF, EF); delay(10);    // avoids crosstalk
  long R = readCm(TR, ER);

  if (L > 30)      { turnLeft();  return; }   // opening on the left
  if (F < 12) {
    if (R > 30)    { turnRight(); return; }
    uTurn(); return;                         // dead end
  }
  float error = TARGET - L;               // + means too close
  integral = constrain(integral + error, -50, 50);
  float derivative = error - lastError;
  lastError = error;
  float corr = Kp * error + Ki * integral + Kd * derivative;
  drive(BASE + corr, BASE - corr);        // steer away / towards
  Serial.println(error);                  // for the Serial Plotter
}
```

Check the sign

Too close to the left wall → error positive → left wheel faster → robot turns right, away from the wall. Draw it to be sure!

Anti-windup

constrain() stops the integral growing forever while the robot is stuck at a corner.

Same loop rate

PID assumes regular timing. Keep delays the same every loop.

MAKE

Code it: part 3

Turns and calibration

```
const int TURN_MS = 380;   // calibrate on YOUR floor!

void turnLeft() {
  drive(BASE, BASE); delay(250);       // move the body past the corner
  drive(-140, 140);  delay(TURN_MS);   // spin about 90 degrees
  drive(BASE, BASE); delay(300);       // enter the new corridor
  integral = 0; lastError = 0;         // fresh start for PID
}
void turnRight() {
  drive(140, -140);  delay(TURN_MS);
  integral = 0; lastError = 0;
}
void uTurn() {
  drive(140, -140);  delay(2 * TURN_MS);
  integral = 0; lastError = 0;
}
```

Calibrate TURN_MS

Put tape on the floor. Change TURN_MS until one spin is a clean 90°. Battery level changes it, so re-check!

Reset PID after turns

Old errors from the last corridor would make the robot jerk in the new one.

Better turns

Turning by time is simple but drifts. Wheel encoders or an MPU6050 gyro make exact turns (challenge!).

Tuning method

Tune like a control engineer

1. **Set Ki = 0, Kd = 0** Start with only P.
2. **Raise Kp** 1, 2, 4, 8... until the robot zig-zags along a straight wall.
3. **Halve Kp** That is your working Kp.
4. **Add Kd** Increase until the zig-zag dies away (try 0.5, 1, 2).
5. **Add a little Ki** Only if the robot sits at a steady 2-3 cm off the target.
6. **Record everything** One change at a time, 3 runs each.

Use the straight corridor

Tune on a 2 m straight wall first. A maze has too many events to judge PID.

Fair test

Same battery level, same start position, same BASE speed for every run.

Record your data

Tuning log

Run	Kp	Ki	Kd	Max error (cm)	Behaviour
Run 1	____	____	____	____	smooth / zig-zag / lazy / crash
Run 2	____	____	____	____	smooth / zig-zag / lazy / crash
Run 3	____	____	____	____	smooth / zig-zag / lazy / crash
Run 4	____	____	____	____	smooth / zig-zag / lazy / crash
Run 5	____	____	____	____	smooth / zig-zag / lazy / crash
Run 6	____	____	____	____	smooth / zig-zag / lazy / crash

Quantify it

Log the error over the Serial Plotter, or count wall touches in 2 m. Numbers beat opinions!

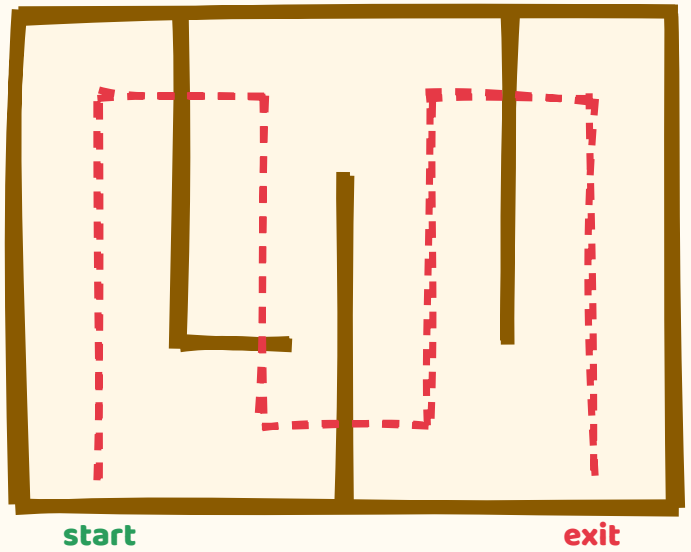
Write a conclusion

Our best values were… because…

Maze race

Solve it, then beat your own time

Version	Run 1	Run 2	Wall touches
Before tuning	____ s	____ s	____
After tuning	____ s	____ s	____
BASE speed + 30	____ s	____ s	____
Final	____ s	____ s	____



Speed vs stability

Faster BASE speed needs different Kp and Kd. Real engineers retune when anything changes.

Rules

Hands off once it starts. 3 lives per team. Fastest clean run wins.

Test like an engineer

Troubleshooting

Symptom	Likely cause / fix
Robot spins in circles	one motor reversed: swap its wires
Motors hum, do not move	BASE too low or batteries flat
Random sharp turns	crosstalk or angled walls: add delays, square walls
Turns into the wall	PID sign reversed: swap + and - corrections
Ignores openings	side threshold (30 cm) too big or too small for your maze

Debug checklist

Debug with wheels lifted and Serial prints of L, F, R and corr.
Change ONE thing at a time.

Film your robot with a phone in slow motion. You will see exactly where it goes wrong.

Level up!

Level up

Choose your challenge

Remember the route: store each turn, delete dead ends, and run the shortest path on the second try (like Micromouse).

Exact turns: use an MPU6050 gyro to turn exactly 90° instead of by time.

PID for speed: use a second PID on the front distance to slow down smoothly before walls.

Centre the corridor: use error = L - R to drive in the exact middle.

Think like an engineer

Our left-hand rule fails in mazes with loops (islands). What could a robot remember to escape them?

Boss level

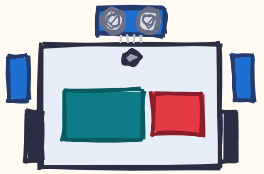
Mark visited junctions: Trémaux's algorithm never gets trapped.

Show what you know

What did we learn?

- [x] Closed-loop control measures, compares and corrects
- [x] P = present error, I = past, D = future trend
- [x] Tune P first, then D, then a little I
- [x] Read sensors one at a time to avoid crosstalk
- [x] Left-hand rule solves any loop-free maze

At home
Notice feedback control at home: a fridge thermostat, an iron, a water tank float valve.



Exit ticket: your robot zig-zags along the wall. Which value do you change first, and how?

Next class: Pick-and-Place Robotic Arm with Record & Replay!